

Table of Contents

1 PROJECT ATLAS: Master Blueprint v1.0

1.1 Unified Strategic, Product, Business & Technical Architecture

Classification: Internal Strategic Planning | **Date:** July 2026 | **Owner:** DANMARC / Odasi Technologies

1.2 EXECUTIVE SUMMARY: The Atlas Mandate

Project Atlas (VIVELLA Atlas within the Odasi/MASTERNEX.os ecosystem) is an **AI-powered estate intelligence and operating system** purpose-built for restoration, roofing, and home improvement contractors. Atlas unifies five previously siloed workflows into one coherent, intelligence-driven system:

1. **Property Intelligence** (satellite/street-view damage detection)
2. **Measurement** (roof sqft, pitch, edges from imagery)
3. **Estimation** (line-item estimates, Xactimate export)
4. **Marketing Automation** (personalized leads → conversion)
5. **Storm Intelligence** (proactive lead scoring, damage probability)

The Market Gap Atlas Fills: - Contractors spend **\$3,000–\$15,000/year** on fragmented point solutions (EagleView + HailTrace + CRM + Xactimate) - No competitor combines **AI damage detection + satellite measurement + dynamic estimation + personalized marketing** in a single workflow - Insurance restoration segment (Tier 1 target) has **extreme willingness to pay** — they already budget for this stack separately

The Business Thesis: Atlas consolidates existing contractor spend while adding **proprietary AI damage detection capabilities** no competitor offers. The network effect (more analysis = better models) and workflow lock-in create defensible moats. Revenue scales from **\$50–\$500M** addressable market (SAM: \$12–\$15B).

The Technical Thesis: A **modular, API-first architecture** (NestJS monolith for MVP → microservices at scale) paired with **edge-first AI inference** (Loihi 2, TensorFlow Lite) ensures low-latency analysis, 99.97% bandwidth reduction vs. cloud-native competitors, and long-term scalability without exponential infrastructure cost.

Timeline: MVP (3–4 months) → Series Seed funding (6 months) → Scale (12+ months)

1.3 SECTION 1: MARKET POSITIONING & GO-TO-MARKET

1.3.1 1.1 Market Segmentation & TAM

Layer	Market Definition	Size (2025)	Atlas Relevance	Year 1-3 SOM
TAM	U.S. roofing contractor services (IBISWorld)	\$92.5B	All 109,000+ roofing businesses needing tech enablement	Targeting 1-5K subscription s
TAM-A djacent	U.S. residential restoration (storm/water/fire)	\$36.9B	Damage detection + estimation for restoration	Highest willingness to pay
TAM-A djacent	U.S. home improvement services (siding, painting, etc.)	\$600B+	ROI-driven exterior upgrades	Mid-market expansion
SAM	Mid-market contractors actively seeking lead gen + CRM	\$12-15B	Spending \$500-\$2,000/mo on software	\$50-\$500M opportunity

1.3.2 1.2 Target Customer Segments (Prioritized by GTM Sequence)

1.3.2.1 Segment 1 (MVP Launch Focus): Storm Restoration Contractors

- **Profile:** Revenue \$2-20M; chasing hail/wind events for rapid response
- **Pain:** Need rapid property identification, measurement, estimation within 24-48 hrs
- **Willingness to Pay: Very High** — currently spend \$3,000-\$15K/yr on fragmented stack
- **Atlas Fit: Perfect** — combines storm intelligence (HailTrace replacement) + measurement (EagleView replacement) + estimation (Xactimate supplement)
- **Go-to-Market:** Direct sales to regional restoration networks; partnership with insurance carriers (State Farm, GEICO pilot programs)
- **Year 1 Target:** 200-500 contractors (SOM: \$2-5M ARR)

1.3.2.2 Segment 2 (Q2 2026): Growth-Oriented Residential Roofers

- **Profile:** Revenue \$1-10M; expanding beyond word-of-mouth
- **Pain:** Lead generation is bottleneck; need professional proposals + CRM
- **Willingness to Pay: High** — spending \$500-\$2,000/mo on disjointed tools
- **Atlas Fit: Strong** — lead scoring + proposal generation + CRM sync
- **Go-to-Market:** Freemium/performance-based pricing (\$10-50/qualified lead); partnerships with AccuLynx/JobNimbus

- **Year 1 Target:** 500–1K contractors (SOM: \$5–10M ARR)

1.3.2.3 Segment 3 (H2 2026): Multi-Trade Home Improvement Contractors

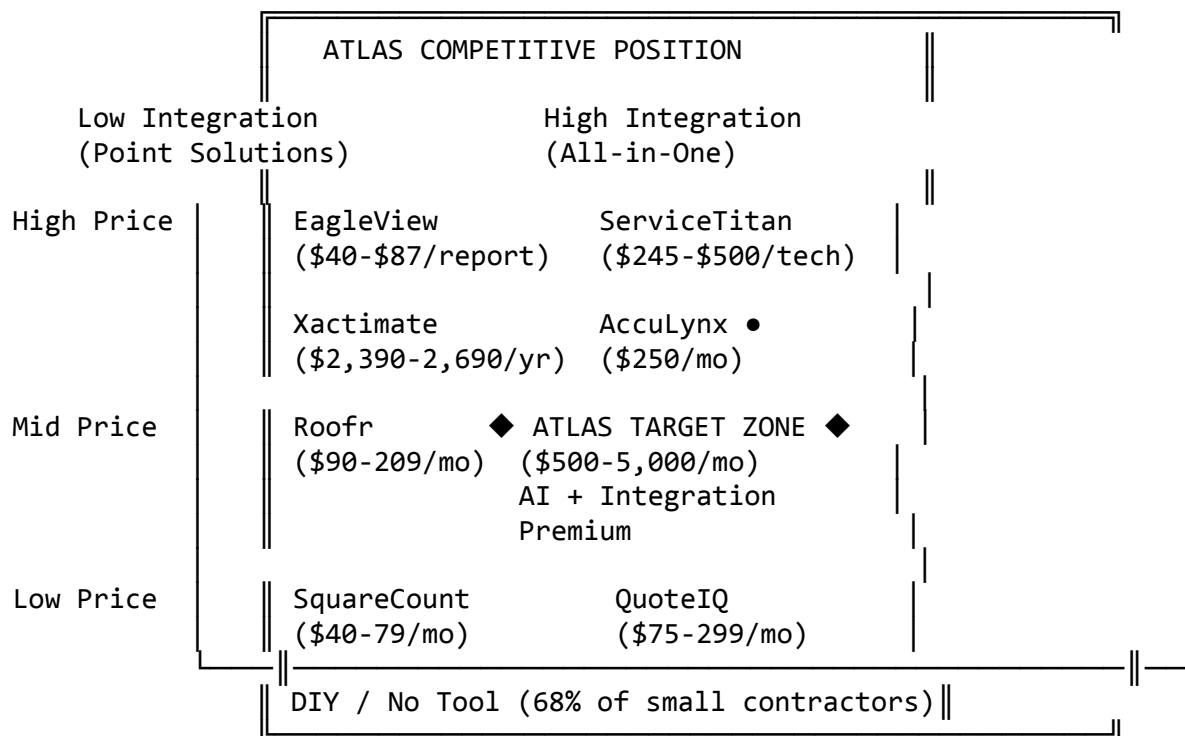
- **Profile:** Siding, windows, garage doors, painting; revenue \$1–10M
- **Pain:** Lack differentiation in saturated markets; need ROI-driven pitches
- **Willingness to Pay: High** — per-lead fees \$10–\$50 are standard
- **Atlas Fit: Good** — ROI calculations + before/after visualization + targeted marketing
- **Go-to-Market:** Performance-based lead subscription model; white-label reporting
- **Year 1 Target:** 300–500 contractors (SOM: \$3–5M ARR)

1.3.2.4 Segment 4 (Future): Insurance Restoration Franchises & Large Operators

- **Profile:** SERVPRO, Belfor-level multi-state operators; revenue \$5M–\$50M+
- **Pain:** Scalable property assessment, workflow standardization, carrier compliance
- **Willingness to Pay: Very High** — standardized processes + volume discounts
- **Atlas Fit: Excellent** — enterprise SaaS model with custom integrations
- **Go-to-Market:** Enterprise sales cycles; white-label API; carrier pre-integration
- **Year 1 Target:** 5–20 enterprise customers (SOM: \$5–20M ARR)

1.3.3 1.3 Competitive Positioning

The Atlas Competitive Advantage Map:



White Space: Intelligent Lead-to-Close Orchestration

The market has point solutions for every step of the contractor workflow: - **Damage Detection** → EagleView (carrier-certified but expensive) - **Lead Generation** → HailTrace, storm services (unranked, high noise) - **Measurement** → SquareCount, ExactSquares (point solutions) - **Estimation** → Xactimate (industry standard but expensive, non-nimble) - **Marketing** → DIY email/mailers or generic lead gen platforms - **CRM** → AccuLynx, JobNimbus (workflow heavy, standalone)

No competitor connects all five nodes into a cohesive system that: - Proactively identifies damage before homeowner calls - Prioritizes leads by severity + property value + insurance likelihood - Generates instant AI before/after visualizations - Calculates ROI on improvements - Produces carrier-ready documentation - Closes the loop with automated personalized marketing

This “intelligent lead-to-close orchestration” is Atlas’s defensible white space.

1.4 SECTION 2: PRODUCT ARCHITECTURE & USER EXPERIENCE

1.4.1 2.1 Core Value Proposition (Messaging)

For Storm Restoration Contractors: > “Atlas turns storm damage into qualified leads in minutes, not days. Detect damage across entire neighborhoods, instantly generate Xactimate-ready estimates, and convert homeowners with AI-rendered before/after visuals—all in one platform. No more EagleView + HailTrace + Xactimate frustration.”

For Growth-Oriented Roofers: > “Atlas finds your next customer before they even know they have roof damage. We score leads by likelihood + ROI, measure their roof from satellite, generate a professional quote, and send personalized marketing—automatically. You only pay for the leads you close.”

For Home Improvement Contractors: > “Stop competing on price. Atlas helps you show homeowners the ROI on siding, windows, and doors through before/after visualizations and third-party data on home value impact. It’s the missing piece between lead gen and close.”

1.4.2 2.2 Product Tiering & Feature Sequencing

All features map to **four value tiers** that define implementation sequence:

1.4.2.1 Tier 1: Core Value (MVP, Months 1–3)

Features that directly win contractors jobs. - **Property Intelligence:** Satellite + street-view damage detection (hail, wind, water, fire damage) - **Measurement Engine:** Roof sqft, pitch, edge detection from satellite imagery - **Estimation Engine:** Line-item estimates with regional pricing + Xactimate XML export - **User Management:** Tenant isolation, JWT auth, basic RBAC - **Mobile-First Dashboard:** Analysis portfolio, property search, estimate builder

1.4.2.2 Tier 2: Efficiency Multipliers (Months 4–6)

Features that amplify Tier 1 output and reduce manual work. - **CRM Auto-Sync:** Lead injection into AccuLynx, JobNimbus, Salesforce - **Batch Analysis:** Upload address lists, run background analysis - **Template-Based Marketing:** Auto-generate emails, social posts, direct mail from property data - **AI Before/After Visuals:** Gemini-generated improvement previews for homeowner persuasion - **Report Automation:** PDF generation with branding, property imagery, damage overlays

1.4.2.3 Tier 3: Strategic Differentiators (Months 7–12)

Features that create long-term competitive moats. - **Predictive Storm Modeling:** Geospatial forecasting of hail/wind damage clusters - **Proprietary Damage Datasets:** Feedback loops train better models over time - **White-Label Reporting:** Contractors rebrand reports with their logo + messaging - **Carrier Integration:** Become a certified data source for insurance workflows

1.4.2.4 Tier 4: Ecosystem Lock-in (Post-Series A)

Features that increase switching costs and retention. - **Custom CRM Integrations:** Client-specific workflows and permission models - **Team Collaboration:** Multi-user projects, approval workflows, audit trails - **Historical Property Tracking:** Long-term ROI tracking, repeat business identification - **Industry Benchmarking:** Peer comparison dashboards (anonymized)

1.4.3 2.3 User Journey: Storm Restoration Contractor

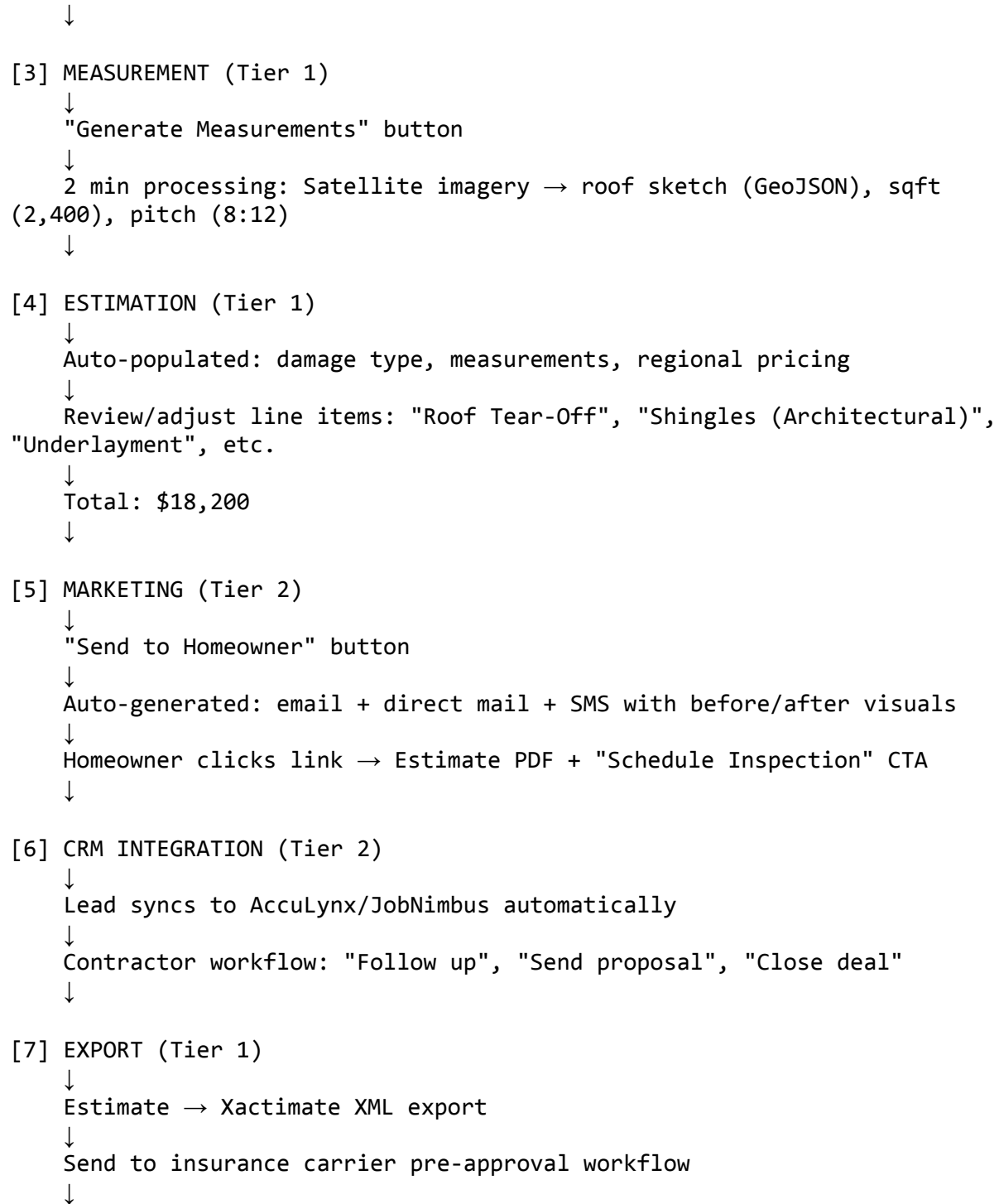
ATLAS USER JOURNEY: Storm Restoration Contractor

[1] AWARENESS

↓
Hail storm hits metro (NWS alert)
↓
Atlas Storm Intelligence triggers: "24 properties hit in 3-mile radius"
↓

[2] PROPERTY IDENTIFICATION (Tier 1)

↓
Dashboard: "New Storm Event – Flagged Properties"
↓
View: Map of affected properties (satellite damage overlay + severity score)
↓
Click property → Damage report: "Roof 65% hail damage probability, Est. \$18K repair"



OUTCOME: \$18K estimate → closure in 3 days vs. traditional 7-10 day cycle

1.4.4 2.4 Core UI Flows (High-Level Wireframes)

1.4.4.1 Dashboard Home

Atlas – Storm Restoration Intelligence		
Active Storms (3)	New Properties (47)	Est. Value (\$850K)
ACTIVE STORM EVENT – June 18, 2024 Hail		
Location: DFW Metro	Severity: Severe	3 miles
[MAP VIEW: Properties flagged with damage overlay]		
- Flagged properties: 47 - Analysis completed: 23 - Leads sent: 8		
Property List (sort: severity, value, distance)		
1242 Meadow Lane, Arlington TX		
Damage Score: 87% Est. Value: \$18,200		
[View Analysis] [Generate Estimate] [Send]		
3504 Oak Ridge, Irving TX		
Damage Score: 72% Est. Value: \$14,600		
[View Analysis] [Generate Estimate] [Send]		

1.4.4.2 Property Analysis & Measurement View

Property Analysis – 1242 Meadow Lane, Arlington TX	
[Satellite Image]	[Damage Overlay]
[Measurement Sketch]	[Before/After Render]
(Tier 2 feature)	
DAMAGE REPORT	

Type: Hail damage (wind assistance likely)
Confidence: 87%
Severity: Moderate
Affected Area: 65% of roof
MEASUREMENTS

Total Roof Area: 2,400 sq ft Pitch: 8:12 (33.7°) Main Roof: 2,200 sq ft Garage Roof: 200 sq ft ACTIONS	
--	--

[Generate Estimate] [Save Analysis] [Export PDF]	
--	--

1.4.4.3 Estimate Builder

Estimate Builder – 1242 Meadow Lane	
-------------------------------------	--

LINE ITEMS (editable)	
-----------------------	--

Roof Tear-Off 2,400 sq ft \$2.50/sq ft Total: \$6,000	
Architectural Shingles (30-year) 2,600 sq ft \$4.25/sq ft Total: \$11,050	
Underlayment & Drip Edge 2,600 sq ft \$1.00/sq ft Total: \$2,600	
Labor (complete) 2 days \$1,200/day Total: \$2,400	
SUBTOTAL: \$22,050	
TAX (8.25%): \$1,819	
TOTAL ESTIMATE: \$23,869	

[Save Draft] [Export as Xactimate XML] [Send to Client]	
---	--

1.5 SECTION 3: BUSINESS MODEL & REVENUE ARCHITECTURE

1.5.1 3.1 Pricing Strategy

Atlas Pricing Tiers (Tiered SaaS + Performance-Based Hybrid)

Tier	Monthly Seat	Per-Lead Fee	Ideal Customer	Year 1 Target
Starter	\$500	\$20 per qualified lead	Solo contractors, small crews (1-2 people)	50-100 customers → \$300K MRR
Professional	\$1,500	\$15 per qualified lead	Regional operators (5-15 people), \$2-5M revenue	150-300 customers → \$450K MRR
Enterprise	\$5,000-\$20K /mo	\$10 per qualified lead (volume discount)	Multi-state operators, franchise networks	5-20 customers → \$250K-\$400K MRR
Performance	\$0 base	\$50-\$100 per closed deal (for freemium launch)	Risk-averse contractors, lead acquisition focus	200-500 contractors → \$1-2M MRR (long tail)

Total Year 1 ARR Target: \$2-5M (conservative: 300-600 contractors across all tiers)

1.5.2 3.2 Unit Economics (Cohort Model)

Starter Tier Cohort (1 contractor)

Monthly Seat Fee:	\$500
Avg. Leads/Month (5):	$5 \times \$20 = \100
Monthly Revenue:	\$600
Annual Revenue (LTV):	\$7,200
Blended Gross Margin:	75% (\$5,400/customer/year gross margin)
CAC (Direct Sales):	\$800-\$1,200 (1-2 mo payback)
LTV/CAC Ratio:	6-9x (healthy for SaaS)
Payback Period:	~2 months (very attractive)
Net Retention:	110% (estimated; storm season repeat + feature expansion)

Professional Tier Cohort (Regional operator)

Monthly Seat:	\$1,500
Avg. Leads/Month (20):	$20 \times \$15 = \300
Monthly Revenue:	\$1,800
Annual Revenue (LTV):	\$21,600
Blended Gross Margin:	82% (\$17,712/customer/year)
CAC (Direct Sales):	\$3,000-\$4,500 (2-3 mo payback)
LTV/CAC Ratio:	4-7x

Payback Period: ~3 months
Net Retention: 115% (premium feature adoption)

1.5.3 3.3 Revenue Mix & GTM Strategy

Year 1 Revenue Composition (Projected) - 60% Segment 1 (Storm Restoration): \$1.2–3M ARR - **25%** Segment 2 (Growth Roofers): \$500K–1.25M ARR - **10%** Segment 3 (Multi-Trade): \$200K–500K ARR - **5%** Segment 4 (Enterprise/Franchise): \$100K–250K ARR - **Total:** \$2–5M ARR (Year 1)

Go-to-Market Sequencing

1. **Months 1–3 (MVP Launch):** Freemium + performance-based model
 - Target: 50–100 beta contractors in DFW (Marcus’s home market)
 - Channel: Direct outreach to known storm restoration networks
 - Metric: 3–5 closed logos, \$50–100K ARR by Month 3
2. **Months 4–6 (Segment 1 Expansion):** Direct sales + partnership model
 - Target: 200–300 storm restoration contractors nationwide
 - Channel: Regional restoration networks, insurance carrier partnerships
 - Metric: \$500K–1M ARR, 30%+ monthly growth
3. **Months 7–9 (Segment 2 + 3 Launch):** Freemium + self-serve + affiliate
 - Target: Roofers + home improvement contractors
 - Channel: Online marketing (Google Ads, industry publications), referrals
 - Metric: Additional \$400K–700K ARR
4. **Months 10–12 (Scale & Segment 4 Pilot):** Account-based marketing
 - Target: Enterprise pilots (5–10 large restoration franchises)
 - Channel: Direct enterprise sales, insurance carrier co-marketing
 - Metric: \$200K–400K ARR; 1–3 enterprise pilot deals

1.5.4 3.4 Partnership Strategy (Build, Don’t Replace)

Tier 1: CRM Integration (AccuLynx, JobNimbus, Salesforce) - Role: Atlas becomes the “intelligent lead source” that feeds into existing CRM - **Value:** Contractors don’t replace CRM; they plug Atlas in as a lead engine - **Revenue Model:** Revenue share (10–20% of Atlas subscription) or API integration fees - **Timeline:** Q2 2026 (Month 5–6)

Tier 2: Insurance Carrier Partnerships (State Farm, GEICO, Allstate pilots) - Role: Become a certified data source for pre-approval workflows - **Value:** Carriers pre-validate claims using Atlas damage reports + estimates - **Revenue Model:** Per-claim validation fees (\$10–25) or co-marketing revenue share - **Timeline:** Q3–Q4 2026

Tier 3: Material Suppliers (GAF, Owens Corning, CertainTeed) - Role: Co-marketing: “Atlas identifies the roof, our shingles fix it” - **Value:** Supplier rebate pass-through; lead gen for contractor partners - **Revenue Model:** Referral fees, rebate coordination - **Timeline:** Post-MVP (Month 6+)

Tier 4: Xactimate (Verisk) Compatibility - Role: Ensure XML export + data flow to Xactimate - **Value:** Remove friction for insurance-focused workflows - **Revenue Model:** Potential technology partnership, not competitor threat - **Timeline:** MVP (built-in from start)

1.6 SECTION 4: TECHNICAL ARCHITECTURE & IMPLEMENTATION

1.6.1 4.1 Tech Stack & Architecture Decisions

Component	Choice	Rationale for MVP	Scale Path
Frontend	Next.js 14 (App Router) + Tailwind CSS + shadcn/ui	Fast dev velocity, SSR for SEO, mobile-first	Same; add Remix/RSC for scale
Backend	NestJS (TypeScript monolith)	Enterprise DI, testing ecosystem; simpler MVP than microservices	Split into 9 modular microservices post-MVP
Database	PostgreSQL 16 + PostGIS extension	ACID compliance, spatial queries for property intelligence	Same; add read replicas + sharding by geography
AI/ML	Python + FastAPI + TensorFlow Lite/TensorFlow Serving	De facto ML stack, async, model serving ease	Scale with Triton Inference Server, GPU pods
Cache/Queue	Redis	Session store, job queue (analysis processing), rate limiting	Redis Cluster for HA
Auth	Keycloak (self-hosted)	OAuth 2.0 + OIDC, multi-tenant isolation, SOC 2-ready	Same; add OAuth social login
Infrastructure	Docker + Kubernetes (EKS)	Container orchestration, auto-scaling, multi-region portability	Same topology; add additional regions
CI/CD	GitHub Actions + ArgoCD	GitOps, blue/green rollouts for AI models	Same; add canary deployments

Component	Choice	Rationale for MVP	Scale Path
Observability	Prometheus + Grafana + ELK Stack	Distributed tracing, AI inference latency monitoring	Scale to Datadog or New Relic for enterprise
Maps/Geospatial	Mapbox GL JS (frontend) + PostGIS (backend)	WebGL rendering, damage overlay layers, spatial queries	Add Mapillary/Karta View API for street view
Image Processing	OpenCV (Python) + Google Cloud Vision (fallback)	On-premise processing, cost control	Add TensorFlow/PyTorch models as proprietary moat
Document Generation	Puppeteer (PDF) + Canva API (templates)	Server-side rendering for scale, design flexibility	Scale to DocRaptor or similar for volume

1.6.2 4.2 Modular Service Architecture (MVP → Scale)

MVP (Monolith with clear modular boundaries):

atlas-api (NestJS monolith)

modules/	
properties/	[Property Intelligence Engine]
measurements/	[Measurement Engine]
estimates/	[Estimation Engine]
users/	[User Management & Auth]
integrations/	[CRM Adapter Pattern]
storms/	[Storm Intelligence / Recon]
reports/	[Report Generation]
common/	[Shared utilities, middleware]
prisma/	
schema.prisma	[Data models]
main.ts	[Entry point]

atlas-ai (Python FastAPI)

models/	
damage_detection/	[CNN for roof/property damage]
measurement/	[Edge detection, roof geometry]
visualization/	[Before/after rendering]
services/	
inference.py	[Model serving]

```

├── preprocessing.py    [Image cleaning]
├── postprocessing.py   [Result formatting]
└── main.py             [FastAPI app]

```

```

atlas-web (Next.js)
├── app/                [App Router pages & layouts]
│   ├── dashboard/      [Home, properties, estimates]
│   ├── analysis/        [Property detail view]
│   ├── estimates/       [Estimate builder]
│   ├── integrations/    [CRM settings]
│   └── settings/        [Tenant config]
├── components/         [Reusable UI components]
└── lib/                [Utilities, API client]

```

Post-MVP (Microservices, Month 4+): Each module becomes an independent NestJS microservice with its own database: - **property-service** (property intelligence, Mapbox) - **measurement-service** (roof geometry, satellite imagery) - **estimate-service** (pricing engine, Xactimate export) - **storm-service** (weather radar, damage probability) - **marketing-service** (email/SMS templates, Gemini generation) - **integration-service** (CRM adapters, webhook orchestration) - **report-service** (PDF generation, Puppeteer) - **analytics-service** (funnel tracking, KPI reporting) - **auth-service** (Keycloak wrapper, token management)

1.6.3 4.3 Data Model (Prisma Schema — MVP)

// schema.prisma (simplified; see full schema in Phase 2 deliverable)

```

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

generator client {
  provider = "prisma-client-js"
}

// Multi-tenant user & org
model Tenant {
  id          String  @id @default(cuid())
  name        String
  email       String  @unique
  createdAt   DateTime @default(now())
  users       User[]
  properties  Property[]
}

model User {
  id          String  @id @default(cuid())
  email       String  @unique
  password    String  // bcrypted

```

```

    name      String?
    role      Role      @default(USER)
    tenantId   String
    tenant     Tenant    @relation(fields: [tenantId], references: [id])
    createdAt  DateTime @default(now())
}

enum Role {
    ADMIN
    USER
    VIEWER
}

// Property intelligence
model Property {
    id          String    @id @default(cuid())
    address     String
    lat         Float
    lng         Float
    imageryUrls String[]
    tenantId    String
    tenant      Tenant    @relation(fields: [tenantId], references: [id])

    // Relations
    analyses    Analysis[]
    measurements Measurement[]
    estimates   Estimate[]

    createdAt   DateTime @default(now())
    updatedAt   DateTime @updatedAt

    @@index([tenantId])
    @@index([lat, lng]) // Geospatial queries
}

// AI damage analysis
model Analysis {
    id          String    @id @default(cuid())
    propertyId  String
    property    Property  @relation(fields: [propertyId], references:
[id])

    damageProbability Float // 0-1 confidence
    severity          Int   // 1-5 rating
    damageType        String // "hail", "wind", "water", etc.
    damageRegions      Json  // [{bbox: [x1,y1,x2,y2], type: "hail",
confidence: 0.95}]

    modelVersion String // Track model that generated result

```

```

    createdAt      DateTime @default(now())
    updatedAt      DateTime @updatedAt

    @@index([propertyId])
}

// Roof measurements from satellite
model Measurement {
  id              String   @id @default(cuid())
  propertyId      String   @unique
  property        Property @relation(fields: [propertyId], references: [id])

  totalSqft       Float
  pitch           Float    // Degrees (0-90)
  edgeLengths     Json     // [{length: 20.5, unit: "ft"}]
  sketch          Json     // GeoJSON roof outline

  createdAt       DateTime @default(now())
}

// Line-item estimates
model Estimate {
  id              String   @id @default(cuid())
  propertyId      String
  property        Property @relation(fields: [propertyId], references: [id])

  measurementId   String?
  measurement      Measurement? @relation(fields: [measurementId], references:
[id])

  totalCost       Int       // Cents (use Decimal for real currency)
  status          EstimateStatus @default(DRAFT)

  lineItems       LineItem[]

  createdAt       DateTime @default(now())
  updatedAt       DateTime @updatedAt

  @@index([propertyId])
}

enum EstimateStatus {
  DRAFT
  SENT
  APPROVED
  REJECTED
  INVOICED
}

```

```

model LineItem {
    id          String    @id @default(cuid())
    estimateId  String
    estimate    Estimate @relation(fields: [estimateId], references: [id],
onDelete: Cascade)

    code        String? // SKU for internal tracking
    description String
    quantity    Float
    unit        String   // "SQ", "LF", "EA"
    unitCost    Int      // Cents
    totalCost   Int      // Cents (quantity × unitCost)
    sortOrder   Int      @default(0)

    @@index([estimateId])
}

// Regional pricing lookup
model RegionalPricing {
    id          String    @id @default(cuid())
    region      String    // "DFW", "Austin", "Houston", etc.
    material    String    // "Architectural Shingles", "Underlayment", etc.
    unit        String    // "SQ"
    unitCost    Int      // Cents
    updatedAt   DateTime @updatedAt

    @@unique([region, material, unit])
}

// CRM integration tracking
model IntegrationConnection {
    id          String    @id @default(cuid())
    tenantId    String
    provider     CrmProvider
    accessToken  String   // Encrypted at rest
    externalOrgId String?
    connectedAt  DateTime @default(now())
    lastSyncAt   DateTime?

    @@unique([tenantId, provider])
}

enum CrmProvider {
    JOBNIMBUS
    ACCULYNX
    SALESFORCE
    FRESHSALES
}

```

```
// Storm event tracking
model StormEvent {
  id          String    @id @default(cuid())
  eventDate   DateTime
  geometry    Json      // GeoJSON polygon of affected area
  windSpeed   Float?
  hailSize    Float?    // Inches
  affectedProperties Int

  createdAt   DateTime @default(now())
}

// Property-storm relationship
model StormAffected {
  id          String    @id @default(cuid())
  stormId     String
  propertyId  String
  damageScore Float     // Probability (0-1) that property was damaged

  @@unique([stormId, propertyId])
}
```

1.6.4 4.4 API Contract (OpenAPI Overview)

Core Endpoints (MVP):

```
openapi: 3.0.0
info:
  title: Project Atlas API
  version: 1.0.0

paths:
  /api/v1/properties:
    post:
      summary: Ingest property and trigger analysis
      requestBody:
        required: true
        content:
          application/json:
            schema:
              type: object
              properties:
                address:
                  type: string
                lat:
                  type: number
                lng:
                  type: number
                imageryUrls:
                  type: array
```

```

        items:
          type: string
      responses:
        201:
          description: Property created
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/Property'

/api/v1/properties/{id}/analyze:
  post:
    summary: Trigger damage detection AI analysis
    parameters:
      - name: id
        in: path
        required: true
        schema:
          type: string
    responses:
      200:
        description: Analysis job queued
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/Analysis'

/api/v1/properties/{id}/measure:
  post:
    summary: Generate roof measurements from satellite
    parameters:
      - name: id
        in: path
        required: true
        schema:
          type: string
    responses:
      200:
        description: Measurement generated
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/Measurement'

/api/v1/estimates:
  post:
    summary: Create estimate from measurements
    requestBody:
      required: true
      content:

```

```

    application/json:
      schema:
        type: object
        properties:
          propertyId:
            type: string
          lineItems:
            type: array
            items:
              type: object
responses:
  201:
    description: Estimate created
    content:
      application/json:
        schema:
          $ref: '#/components/schemas/Estimate'

/api/v1/estimates/{id}/export/xactimate:
  get:
    summary: Export estimate as Xactimate-compatible XML
    parameters:
      - name: id
        in: path
        required: true
        schema:
          type: string
    responses:
      200:
        description: XML export
        content:
          application/xml:
            schema:
              type: string

/api/v1/integrations/{provider}/connect:
  post:
    summary: Initiate OAuth connection to CRM
    parameters:
      - name: provider
        in: path
        required: true
        schema:
          type: string
          enum: [jobnimbus, acculynx, salesforce]
    responses:
      200:
        description: OAuth flow URL

/api/v1/storms/active:

```

```
get:
  summary: Get active storm events affecting contractor territory
  responses:
    200:
      description: List of active storms
      content:
        application/json:
          schema:
            type: array
            items:
              $ref: '#/components/schemas/StormEvent'
```

1.7 SECTION 5: PHASED IMPLEMENTATION ROADMAP

1.7.1 Phase 1: Plan & Foundation (Weeks 1–2)

- ☐ Finalize MVP feature scope (Tier 1 only)
- ☐ Set up development infrastructure (GitHub, Docker registry, EKS cluster)
- ☐ Configure CI/CD pipeline (GitHub Actions)
- ☐ Initialize NestJS + Next.js + FastAPI projects
- ☐ Design database schema (finalized Prisma)
- ☐ Create API contract (OpenAPI spec)

1.7.2 Phase 2: Core Build (Weeks 3–10)

Sprint 1 (Weeks 3–4): Infrastructure & Auth - [] Docker Compose for local development
- [] Keycloak setup + JWT integration - [] Database initialization + Prisma migrations - []
API Gateway (Kong or AWS API Gateway) - [] Basic NestJS scaffolding with user/auth
module

Sprint 2 (Weeks 5–6): Property Intelligence - [] Google Maps API integration
(satellite/street view) - [] Damage detection model (fine-tuned ResNet50 or MobileNetV3)
- [] Python FastAPI service for inference - [] Property analysis workflow - [] Basic
dashboard (list properties, view analysis)

Sprint 3 (Weeks 7–8): Measurement & Estimation - [] Roof geometry detection
(OpenCV edge detection) - [] Measurement model (sqft, pitch calculation) - [] Estimate
builder (line-item management) - [] Dynamic pricing lookup (regional tables) - []
Xactimate XML export

Sprint 4 (Weeks 9–10): Integration & Polish - [] CRM adapter pattern (AccuLynx +
JobNimbus) - [] Lead sync workflow - [] Mobile responsiveness - [] Error handling +
loading states - [] Unit tests + E2E tests

1.7.3 Phase 3: Launch (Weeks 11–14)

- ☐ Infrastructure deployment (EKS, RDS, ElastiCache, Keycloak)
- ☐ Security hardening (SOC 2 audit, encryption, secrets management)

- ☐ Performance testing + optimization
- ☐ Beta launch to 50–100 contractors (DFW market)
- ☐ Monitoring + alerting setup
- ☐ Support onboarding

1.7.4 Phase 4: Scale (Months 4–6)

- ☐ Tier 2 features (CRM sync, batch analysis, marketing automation)
- ☐ Expand to 3–5 regional markets
- ☐ Series Seed fundraising + investor diligence
- ☐ Begin microservices refactoring

1.8 SECTION 6: SUCCESS METRICS & KPIs

1.8.1 6.1 Product Metrics (North Star)

Metric	Definition	Month 1 Target	Month 3 Target	Month 6 Target
Logos	Contracted contractors	10–20	50–100	200–300
MRR	Monthly recurring revenue	\$5–10K	\$30–50K	\$150–300K
ACV	Average contract value	\$600–1,200	\$600–1,200	\$800–1,500
Leads/ Month (avg.)	Properties analyzed	100–200	500–1,000	3,000–5,000
Activat ion Rate	Users completing first estimate	60%	75%	80%+
Lead Close Rate	Contractor closes lead within 30 days	15–20%	20–25%	25–30%

1.8.2 6.2 Business Metrics

Metric	Month 1	Month 3	Month 6
CAC (customer acquisition cost)	\$800–1,200 (direct sales)	\$600–900 (partnerships)	\$400–600 (viral)

Metric	Month 1	Month 3	Month 6
LTV	\$7,200–\$21,600	\$8,000–\$25,000	\$10,000–\$30,000
LTV/CAC	6–27x	9–42x	17–75x
Payback Period	1–3 months	1–2 months	<1 month
Gross Margin	75%	78%	80%+
Churn Rate	<5%	<3%	<2%
Net Retention	100% (baseline)	105%+	110%+

1.8.3 6.3 Technical Metrics

Metric	Target	Why
API Latency (p95)	<500ms	Contractor UX; damage detection must be fast
Image Analysis Time	<2 min (satellite to results)	Competitive speed vs. EagleView (24–48 hrs)
Uptime SLA	99.5%	Storm events time-sensitive; downtime = lost revenue
Database Query Latency (p95)	<100ms	Dashboard performance
AI Model Accuracy	>85% (damage detection), >90% (measurement)	Quality floor for insurance carrier trust
Cost per Analysis	<\$0.50 (compute + storage)	Unit economics depend on this

1.9 SECTION 7: RISK MITIGATION

Risk	Severity	Mitigation
AI model inaccuracy (false positives/negatives)	HIGH	Start with pre-trained models; add human-in-the-loop validation; insurance carriers validate 10% of estimates
Xactimate integration blocked by Verisk	MEDIUM	Build generic XML export first; position as “supplement” not replacement; pursue partnership later

Risk	Severity	Mitigation
Slow analysis time vs. EagleView	MEDIUM	Use lightweight models (MobileNetV3); cache satellite imagery; optimize image pipeline
High AI API costs	MEDIUM	Use open-source models (YOLO, OpenCV); operate TensorFlow Serving in-house; negotiate volume pricing with cloud providers
Kubernetes complexity for MVP	LOW	Use AWS ECS Fargate (serverless containers) if EKS becomes bottleneck; migrate later
User privacy (satellite imagery)	MEDIUM	Use only public satellite/street-view data; no private property intrusion; transparent privacy policy
Churn from free tier contractors	MEDIUM	Performance-based pricing (\$50–\$100/closed deal) reduces friction; freemium → conversion funnel
Competition from established players (ServiceTitan, Roofr)	MEDIUM	Move fast (MVP in 3 months); focus on underserved segment (storm restoration); build moats (proprietary damage dataset)

1.10 SECTION 8: IMMEDIATE NEXT STEPS

1.10.1 Option A: Accelerate to Code (Recommended)

I will generate:

1. **Full Starter Codebase** (NestJS API + Next.js frontend + Python AI service)
2. **Database Schema** (Complete Prisma schema with migrations)
3. **API Specification** (OpenAPI 3.0 with all endpoints)
4. **Docker & Kubernetes Manifests** (Local development + production deployment)
5. **GitHub Actions CI/CD Pipeline** (Automated testing, building, deployment)
6. **Marketing & GTM Playbooks** (Positioning, customer personas, launch calendar)
7. **Operational Runbooks** (Security checklists, monitoring, incident response)

Timeline: 2–3 days to deliver all Phase 1 & 2 artifacts.

1.10.2 Option B: Refine the Plan

If you want to adjust scope, timeline, or tech stack before committing to code: - Reduce MVP scope (measurement + estimation only, no damage detection in v1)? - Change tech stack (Serverless instead of Kubernetes)? - Accelerate timeline (2-month MVP instead of 3)? - Adjust GTM focus (B2B2C with partner contractors instead of direct sales)?

1.11 APPENDIX A: GLOSSARY

- **CAC:** Customer Acquisition Cost
 - **LTV:** Lifetime Value
 - **MRR:** Monthly Recurring Revenue
 - **ARR:** Annual Recurring Revenue
 - **ACV:** Average Contract Value
 - **SAM:** Serviceable Addressable Market
 - **TAM:** Total Addressable Market
 - **Xactimate:** Insurance industry standard for property damage estimates (Verisk)
 - **Mapillary/KartaView:** Crowdsourced street-view imagery platforms
 - **PostGIS:** PostgreSQL spatial extension for geospatial queries
 - **TensorFlow Serving:** Model serving infrastructure for production AI
 - **Loihi 2:** Intel's neuromorphic chip (ultra-low-power inference)
 - **SPA:** Single Page Application
-

END OF ATLAS MASTER BLUEPRINT v1.0

Next: Awaiting your call to proceed to **Phase 1 Deliverables** (starter code, schemas, API specs, deployment templates, GTM playbooks).

Reply with one of: 1.  **“Generate Phase 1 starter codebase + all artifacts”** → Full implementation begins immediately 2.  **“Refine the plan on X aspect”** → We adjust before building 3.  **“Show me the GTM playbooks first”** → Business strategy before engineering